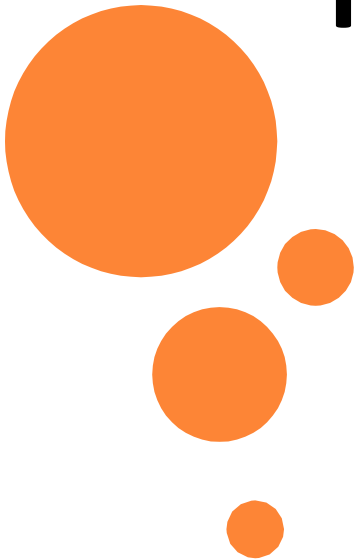


SEMÁNTICA OPERACIONAL

TIPO DE DATOS



TIPOS DE DATOS

- CONCEPTO
- HISTORIA
- TIPOS DE DATOS
- TIPOS PREDEFINIDOS
- TIPOS DEFINIDOS POR EL USUARIO
- CONSTRUCTORES
- PUNTEROS Y MANEJO DE MEMORIA
- TADs
- SISTEMA DE TIPOS



TIPOS DE DATOS - CONCEPTO

Lenguajes de Programación

- Organización de los datos a través del concepto de tipo de datos.
- Forma de clasificar datos de acuerdo con diferentes categorías.
- Más que un conjunto de datos pues tienen un comportamiento semántico o sentido.

Podemos definir a un tipo de datos como un conjunto de valores y un conjunto de operaciones que se pueden utilizar para manipularlos.



TIPOS DE DATOS - HISTORIA

- Los primeros lenguajes tenían el inconveniente de que las estructuras de datos podían ser modeladas sólo con los pocos **tipos de datos básicos definidos por el lenguaje**.
- Se empieza a ver una clara intención de soportar **varios y distintos tipos de datos** con el objeto brindar un mayor apoyo al desarrollo de una amplia variedad de aplicaciones. (**Legibilidad – Modificabilidad**)
- Tomando el concepto de **tipo de dato definido por el usuario** arribamos al concepto de **tipo de dato abstracto**. Separa la representación y conjunto de operaciones (invisibles al usuario).
- La evolución del TAD es el concepto de **Clase**.



TIPOS DE DATOS - CATEGORÍAS

	ELEMENTALES	COMPUESTOS
PREDEFINIDOS	ENTEROS	STRING
	REALES	
	CARACTERES	
	BOOLEANOS	
DEFINIDOS POR EL USUARIO	ENUMERADOS	ARREGLOS
		REGISTROS
		LISTAS
DOMINIO DE UN TIPO ---->		VALORES POSIBLES

TIPOS DE DATOS - CATEGORÍAS

- Cualquier lenguaje de programación posee un conjunto finito de **tipos predefinidos** (built-in/primitivos), que normalmente **reflejan el comportamiento del hardware subyacente**. A nivel de hardware, los valores pertenecen al dominio sin tipo, lo que constituye el dominio universal, estos; son interpretados de manera diferente, según las. Por ej. El byte "01001010" puede ser el 74 o la "J"
- Los Lenguajes de programación permiten al programador **especificar agrupaciones de objetos de datos elementales** (o tipos predefinidos).
- Esto se logra mediante una serie de **constructores** que permiten definir lo que denominamos **tipo de dato definido por el usuario**.



TIPOS DE DATOS - TIPOS PREDEFINIDOS

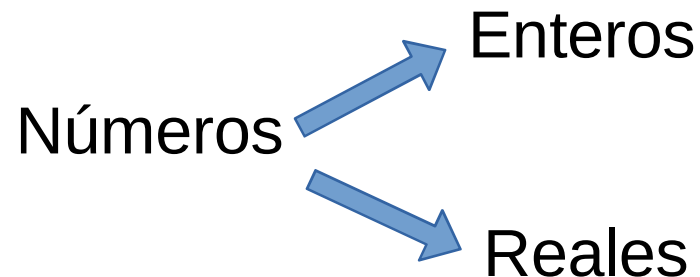
TIPOS PREDEFINIDOS

- Reflejan el comportamiento del hardware subyacente y son una abstracción de él.
- Las ventajas de los tipos predefinidos son:
 - Invisibilidad de la representación
 - Verificación estática
 - Desambiguar operadores
 - Control de precisión



TIPOS DE DATOS - TIPOS PREDEFINIDOS

TIPOS PREDEFINIDOS



Valores

Longitud Máxima

Caracteres

Conversiones

Booleano

Que un conjunto de valores de un tipo sea definido por la implementación del lenguaje significa que será seleccionado por el compilador, mientras que si el tipo es definido por el lenguaje será definido en su definición.



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

TIPOS DEFINIDOS POR EL USUARIO

Los Lenguajes de programación permiten al programador **especificar agrupaciones de objetos de datos elementales** (o tipos predefinidos) y, de forma recursiva, agregaciones de agregados. Esto se logra **mediante** la prestación de una serie de **constructores** que permiten definir lo que denominamos **tipo de dato definido por el usuario**.

Separan la especificación de la implementación. Se definen los tipos que el problema necesita.

- Definir nuevos tipos e instanciarlos
- Chequeo de consistencia



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Legibilidad : elección apropiada de nuevos Nombres

dias [0..31]

vec [dias] vec[0..31]

Estructura jerárquica de las definiciones de tipos: proceso de refinamiento

Record Persona {

String nombre, apellido;

int edad;

..... }

Persona vecino= new Persona(...);

vecinos = array[1..10] of Persona;

Modificabilidad : Solo se cambia en la definición.

Factorización: Se usa la cantidad de veces necesarias.

Abstracción: La instanciación de los objetos en un tipo dado implica una descripción abstracta de sus valores. Los detalles de la implementación solo quedan en la definición del tipo.

TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

COMPUESTOS – CONSTRUCTORES

- Producto Cartesiano
- Correspondencia Finita
- Unión
- Recursión



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Producto Cartesiano

El **producto cartesiano** de n conjuntos $A_1, A_2, \dots, A_n$, denotado $A_1 \times A_2 \times \dots \times A_n$, es un conjunto cuyos elementos están ordenados n -tuplas $(a_1, a_2, \dots, a_n)$, donde cada a_k pertenece a A_k .

Por ejemplo, un polígono puede ser descritos por:

- un número entero (sus lados) y
- un real (longitud de cada borde).

De esta forma el polígono sería un elemento del conjunto del producto cartesiano entre el conjunto de los enteros y el conjunto de los reales.

Estructura de datos-----> Estructuras / Registro



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Producto Cartesiano

- C: estructuras

```
typedef struct {
```

Definición

```
    int nro_lados;
```

Campos

```
    float tamaño_lado;
```

```
} reg_poligon;
```

```
reg_poligon pol = {3,3.45};
```

Instancia con
valor compuesto
inicial

```
pol.nro_lados = 4;
```

Acceso
mediante
notación
puntual



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Correspondencia Finita

Correspondencia finita es una función de un conjunto finito de valores de un tipo de **dominio DT** en valores de un tipo del **dominio RT**.

correspondencia finita en general

$$f: DT \longrightarrow RT$$

Si DT es un subrango de enteros

$$f: [li..ls] \longrightarrow RT$$

conjunto de valores accesibles via un subinidice

Por ejemplo arreglos, colecciones, diccionarios



TIPOS DE DATOS - MANEJO DE MEMORIA

Correspondencia Finita

C (arreglos):

char digits [10]; ← Definición

[0..9] ↔ char ← Rango del índice / Tipo del dominio

for (i=0; i < 10; ++i) ← Tipo del resultado

digits[i] = "; ← Acceso

Phyton:

l = ["una lista", [1, 2]]

mi_var = l[1][0] # mi_var vale 1



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Unión y Unión Discriminada

La unión / union discriminada de dos o mas tipos define un tipo como la disjunción de los tipos dados.

- Permite manipular diferentes tipos en distinto momento de la ejecución
- Chequeo dinámico
- La declaración es muy similar a la del producto cartesiano. La diferencia es que sus campos son mutuamente excluyentes.



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Unión y Unión Discriminada

Permite almacenar diferentes tipos de datos en la **misma ubicación de memoria**.

Su característica principal es que todos sus miembros comparten el mismo espacio, por lo que, en cualquier momento, solo puede contener un valor válido.

```
union address{//campos mutuamente exclusivos
```

```
    short int offset;
```

```
    long unsigned int absoluto;
```

```
};
```



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Unión y Unión Discriminada

Se reserva sólo el espacio del mayor de los campos.

Los datos se almacenan comenzando por la misma posición de memoria y con la estructura del campo seleccionado.

Esta construcción no guarda en ningún lugar cuál de los campos se está utilizando.

Si esa información es necesaria, el programador debe almacenarla



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Unión y Unión Discriminada

Los valores del tipo address deben ser tratados en forma distinta si son de un tipo o del otro.

```
enum descriptor {abs,off}
```

```
typedef struct {
```

```
    address location;
```

```
    descriptor kind;
```

```
}address_seguro;
```

En C

Queda en manos del programador controlar el la opción elegida



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Unión y Unión Discriminada

Unión discriminada agrega un discriminante para indicar la opción elegida.

Si tenemos la unión discriminada de dos conjuntos S y T, y aplicamos el discriminante a un elemento e perteneciente a la unión discriminada devolverá S o T.

- El elemento e debe manipularse de acuerdo al valor del discriminante.
- En la unión y en la unión discriminada el chequeo de tipos debe hacerse en ejecución.
- La unión discriminada se puede manejar en forma segura consultando el discriminante antes de utilizar el valor del elemento.

TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Unión y Unión Discriminada

```
// 1. Definir los tipos específicos
interface Circle {
  kind: "circle"; // Discriminante
  radius: number;
}

interface Square {
  kind: "square"; // Discriminante
  side: number;
}

// 2. Crear la unión discriminada
type Shape = Circle | Square;

// 3. Función que utiliza el discriminante
function calculateArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      // TypeScript sabe que 'shape' es Circle
      return Math.PI * shape.radius ** 2;
    case "square":
      // TypeScript sabe que 'shape' es Square
      return shape.side * shape.side;
  }
}
```

En
Typescript



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Unión y Unión Discriminada

Problemas:

- El discriminante y las variantes pueden manejarse independientemente uno de otros.
- La implementación del lenguaje puede ignorar los chequeos.
- Puede omitirse el discriminante, con lo cual aunque se quisiera no se puede chequear.



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Recursión

Un tipo de dato recursivo T se define como una estructura que puede contener componentes del tipo T.

Define datos agrupados:

- cuyo tamaño puede crecer arbitrariamente
- cuya estructura puede ser arbitrariamente compleja.



TIPOS DE DATOS - DEFINIDOS POR EL USUARIO

Recursión

- Los lenguajes de programación convencionales soportan la implementación de los tipos de datos recursivos a través de los punteros.
- Un **puntero** es una referencia a un objeto.
- Una **variable puntero** es una variable cuyo r-valor es una referencia a un objeto.
- Es una estructura **autoreferenciada** es una variable de tipo struct en la que uno o más de sus campos son punteros del mismo tipo que la estructura que lo define. Bajo este concepto, una variable definida así puede enlazarse (apuntar), a través de estos campos, a otra u otras variables del mismo tipo, generando las denominadas "Listas ligadas", "Árboles", etc.

TIPOS DE DATOS – PUNTEROS Y MANEJO DE MEMORIA

Estructuras de tamaño arbitrario: permite número de items no determinado: los punteros permiten conectar juntos muchos items sin tener un nombre explícito para todos ellos.
(recursión)

Relaciones múltiples entre los items: los punteros permiten que el dato sea puesto en varias estructuras sin necesidad de duplicarlo.

Acceso a bajo nivel: los punteros están cerca de la máquina

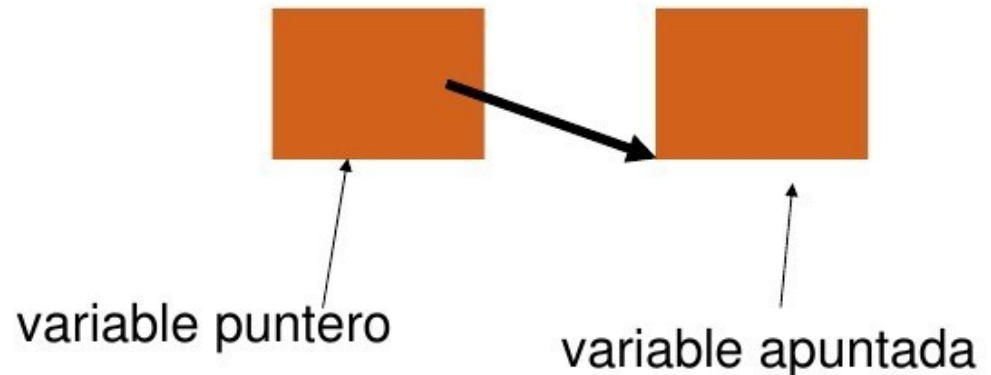


TIPOS DE DATOS - PUNTEROS

Recursión

VALORES:

- direcciones de memoria
- valor nulo (no asignado)



OPERACIONES (l-valor r-valor r-valor de la variable apuntada)

- **Asignación de valor:** generalmente asociado a la alocaión de la variable apuntada
- **Referencias:** a su valor (como dirección), operaciones entre punteros al valor de la variable apuntada: desreferenciación implícita



TIPOS DE DATOS - PUNTEROS

Los punteros son un mecanismo muy potente para definir estructuras de datos recursivas.

Por acceder a bajo nivel, pueden oscurecer o hacer **inseguros** a los programas que los usan.

Se compara a los punteros con los *go to*:

- Los *go to* amplían el rango de sentencias que pueden ejecutar.
- Los punteros amplían el rango de las celdas de memoria que pueden ser referenciadas por una variable y también amplían el tipo de los valores que puede contener un objeto.



TIPOS DE DATOS - PUNTEROS

INSEGURIDAD DE LOS PUNTEROS

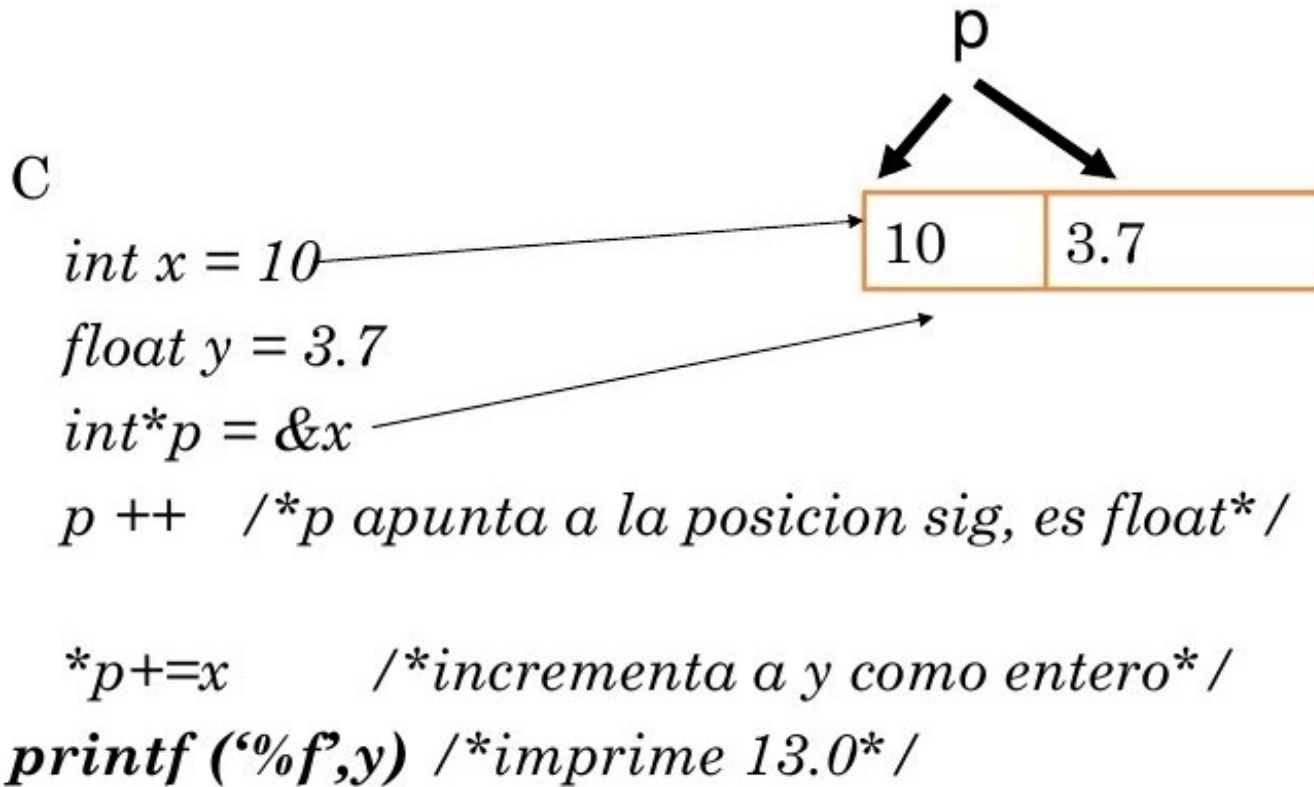
1. Violación de tipos
2. Referencias sueltas – referencias dangling
3. Punteros no inicializados
4. Punteros y uniones discriminadas
5. Alias
6. Liberación de memoria: objetos perdidos



TIPOS DE DATOS - PUNTEROS

INSEGURIDAD DE LOS PUNTEROS

1. Violación de tipos



TIPOS DE DATOS - PUNTEROS

INSEGURIDAD DE LOS PUNTEROS

2. Referencias sueltas – referencias dangling

Si este objeto no está alocado se dice que el puntero es peligroso (dangling).

Una referencia suelta o dangling es un puntero que contiene una dirección de una variable dinámica que fue desalocada. Si luego se usa el puntero producirá error.



TIPOS DE DATOS - PUNTEROS

INSEGURIDAD DE LOS PUNTEROS

3. Punteros no inicializados

Peligro de acceso descontrolado a posiciones de memoria

Verificación dinámica de la inicialización

Solución: valor especial nulo:

- nil en Pascal
- void en C/C++
- null en ADA,
- none en Phyton



TIPOS DE DATOS - PUNTEROS

INSEGURIDAD DE LOS PUNTEROS

4. Punteros y uniones discriminadas

```
union ojo{  
  int int_var  
  int* int_ref}
```

En el caso de C, este es el mismo efecto que causa la aritmética de punteros.

Para resolver este problema asociado con los punteros Java elimina la noción de puntero explícito completamente.



TIPOS DE DATOS - PUNTEROS

INSEGURIDAD DE LOS PUNTEROS

4. Punteros y uniones discriminadas

```
union MiUnion {
    int* pInt;
    float* pFloat;
};

int main() {
    int valor = 5;
    union MiUnion u;
    u.pInt = &valor;

    printf("%p\n", (void*)u.pInt);

    // Ahora cambiamos el miembro activo
    u.pFloat = NULL;

    // PELIGRO: u.pInt ya no es seguro usarlo, y si se hace,
    // puede referenciar memoria liberada o nula.
    return 0;
}
```



TIPOS DE DATOS - PUNTEROS

INSEGURIDAD DE LOS PUNTEROS

5. Alias

`int* p1`

`int* p2`

`int x`

`p1 = &x`

`p2 = &x`

p1 y p2 son punteros
p1 y x son alias
p2 y x también lo son



TIPOS DE DATOS - PUNTEROS

INSEGURIDAD DE LOS PUNTEROS

6. Liberación de memoria: objetos perdidos

Las variables puntero se alocan como cualquier otra variable en la pila de registros de activación.

- Los objetos (apuntados) que se alocan a través de la primitiva **new** son alocados en la heap.
- La memoria disponible (**heap**) podría rápidamente agotarse a menos que de alguna forma se devuelva el almacenamiento alocado liberado.



TIPOS DE DATOS – MANEJO DE MEMORIA

Si los objetos en el heap dejan de ser accesibles esa memoria podría liberarse

garbage (objetos perdidos)

Un objeto se dice accesible si alguna variable en la pila lo apunta directa o indirectamente.

Un objeto es basura si no es accesible.

Mecanismos para desalocar memoria

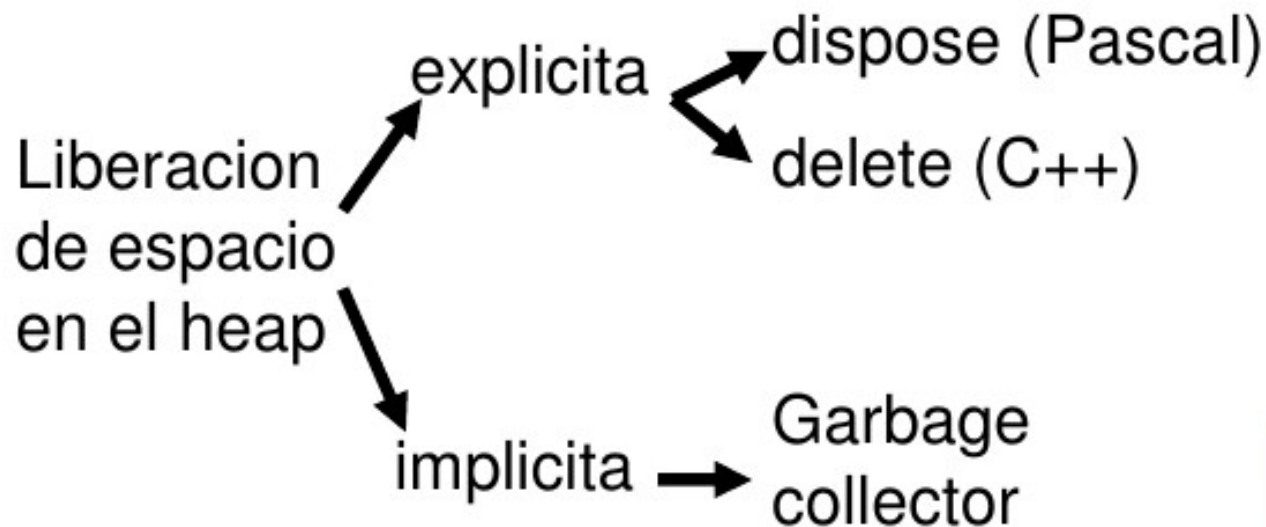


TIPOS DE DATOS – MANEJO DE MEMORIA

Liberación de Memoria

reconocimiento de que
porción de la memoria es
basura

se requiere o no
intervención del usuario



Responsabilidad
del programador

Independiente de
la aplicación

TIPOS DE DATOS – MANEJO DE MEMORIA

El reconocimiento de la basura recae en el programador, quien notifica al sistema cuando un objeto ya no se usa.

No garantiza que no haya otro puntero que apunte a esta dirección definida como basura, este puntero se transforma en dangling. (puntero suelto).

Este error es difícil de chequear y la mayoría de los lenguajes no lo implementan por que es costoso.

TIPOS DE DATOS – MANEJO DE MEMORIA

● Liberación de Memoria

EXPLICITA: PASCAL

- Proceso llamado *dispose* libera memoria en el heap
- Puede generar referencias sueltas
- Para evitarlo se necesitaría una verificación dinámica para garantizar el uso correcto del *dispose*

new (p)

q := *p*

dispose (p)

...*q* ...

libera el espacio del nodo
y pone p en nil

q referencia
suelta

TIPOS DE DATOS – MANEJO DE MEMORIA

● Liberación de Memoria

Implícita

El sistema, durante la ejecución tomará la decisión de descubrir la basura por medio de un algoritmo de recolección de basura: garbage collector.

Muy importante para los lenguajes de programación que hacen un uso intensivo de variables dinámicas.(LISP, Python).

TIPOS DE DATOS – MANEJO DE MEMORIA

● Liberación de Memoria

Implícita

Se ejecuta durante el procesamiento de las aplicaciones

Sistema interactivo o de tiempo real: no bajar en el rendimiento y evitar los riesgos.

Debe ser muy
eficiente

Debe ejecutar en
paralelo

TIPOS DE DATOS – MANEJO DE MEMORIA

● Liberación de Memoria

Implícita

- En Algol-68 y Simula 67 el garbage collector reclama automáticamente la memoria no utilizada.
- Eiffel y Java que uniformemente tratan a todos los objetos como referenciados por punteros, proporcionan un recolector automático.
- ADA chequea dinámicamente que el tiempo de vida de los objetos apuntados sea menor igual que el del puntero.
- PHP chequea una cuenta de referencias al objeto.

TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

Causas: Las variables de tipo puntero se asignan como cualquier otra variable en la pila de registros de activación. Los objetos de datos a los que apuntan se asignan en tiempo de ejecución a la Heap. De hecho, la memoria libre podría agotarse rápidamente, a menos que haya alguna forma de devolver el almacenamiento asignado al área libre. La memoria asignada puede reutilizarse el objeto ya no es accesible. Se dice que dicho objeto es basura. Un objeto es basura si ninguna variable en la pila apunta a él y, recursivamente, ninguna otra basura apunta a él.

Debemos:

- Reconocer que alguna porción de la memoria asignada es basura
- Liberar la memoria de objetos basura para asignar ante nuevas peticiones de memoria para alocar objetos.

TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

Estrategias y Versiones

Podemos mencionar dos grandes estrategias que resuelven los puntos anteriores en términos generales.

- **Conteo de Referencias:** cada objeto asignado a la Heap tiene un campo descriptor para almacenar el número actual de punteros a él. Cada vez que una variable de tipo puntero se desaloca de la pila, el conteo de referencias del objeto se reduce en uno. Cuando el conteo de referencias se vuelve cero, el objeto se declara como basura, y el conteo de referencias de cualquier objeto apuntado por él se reduce en uno. Este método libera un objeto tan pronto como se descubre que ya no tiene referencias. El problema con este método, sin embargo, es que no funciona para estructuras de datos circulares (como una lista circular). Si se desasigna un puntero a la cabeza de una lista circular, los nodos de la lista no se consideran basura, porque el conteo de referencias para cada nodo es 1.

TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

Estrategias y Versiones

Podemos mencionar dos grandes estrategias que resuelven los puntos anteriores en términos generales.

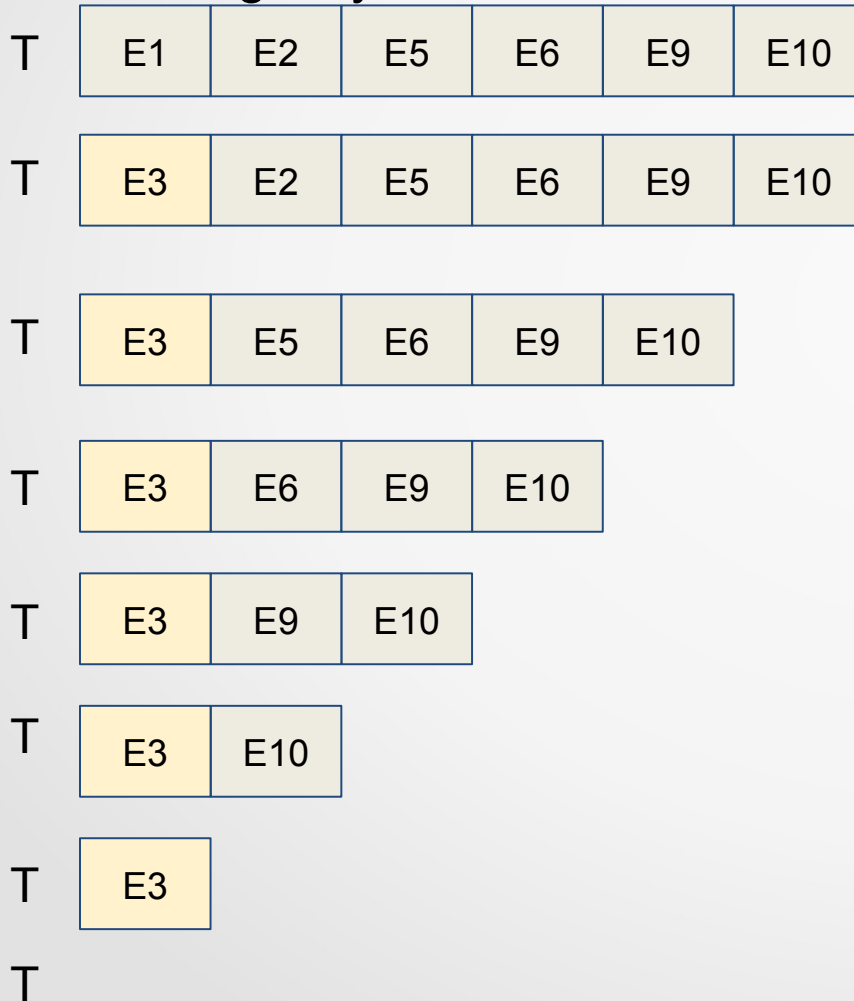
- **Recolección por Trazado:** Se asignan celdas libres hasta que el espacio libre se agote o llegue a una determinada cantidad. Solo en ese momento el sistema entra en una fase de recolección de basura. Este esquema trabaja bajo la suposición simplificadora de que:
 - los objetos tienen un tamaño fijo
 - se sabe qué campos de un objeto contienen punteros a otros objetos de datos de la heap
 - es posible encontrar todos los punteros desde la pila hacia la Heap.

Inicialmente se usa un conjunto de trabajo de punteros T que contiene los valores de la pila que apuntan a la heap. Si un elemento E de la Heap es referenciado, se marca, así como sus objetos referenciados. E se reemplaza por los punteros a los nodos contenidos en E, si no están marcados. Cuando T se vacía, todos los objetos de la heap alcanzables han sido marcados. Todos los demás objetos son basura. En resumen se recorre el listado de objetos referenciados y los que no lo son, se eliminan.

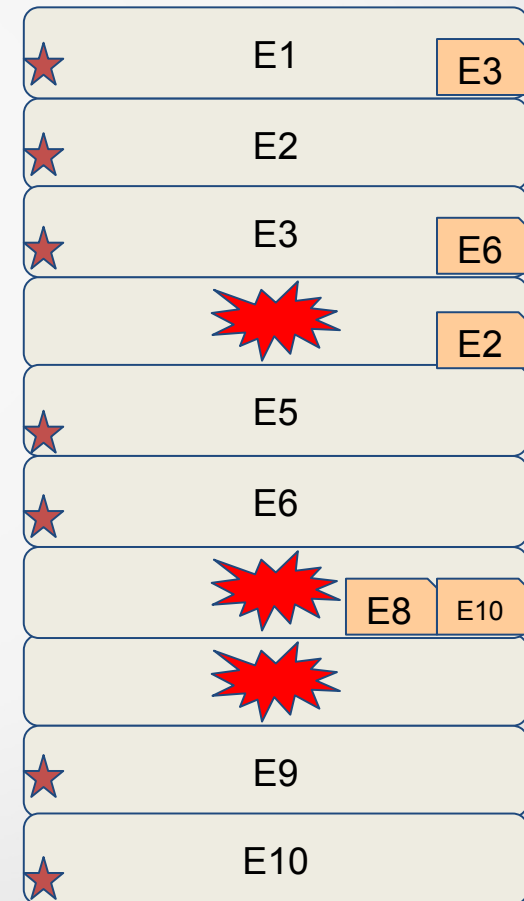
TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

Estrategias y Versiones.



HEAP



TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

Estrategias y Versiones.

Una desventaja de la estrategia anterior es que mínimamente se necesita una estructura cuya profundidad máxima podría ser proporcional a la cadena más larga a través del heap. En la práctica, el espacio para esta pila puede no estar disponible: después de todo, ejecutamos la recolección de basura cuando estamos a punto de quedarnos sin espacio.

Una implementación alternativa de la estrategia anterior denominada (Inversión de Punteros) utiliza una técnica para incrustar el equivalente de la pila en campos ya existentes en los bloques del heap. Más específicamente, a medida que el recolector explora el camino hacia un bloque determinado, invierte los punteros que sigue, de modo que cada uno apunte hacia el bloque anterior en lugar de hacia el siguiente. De esta forma se advierte prematuramente cuando un objeto de la heap no es referenciado.

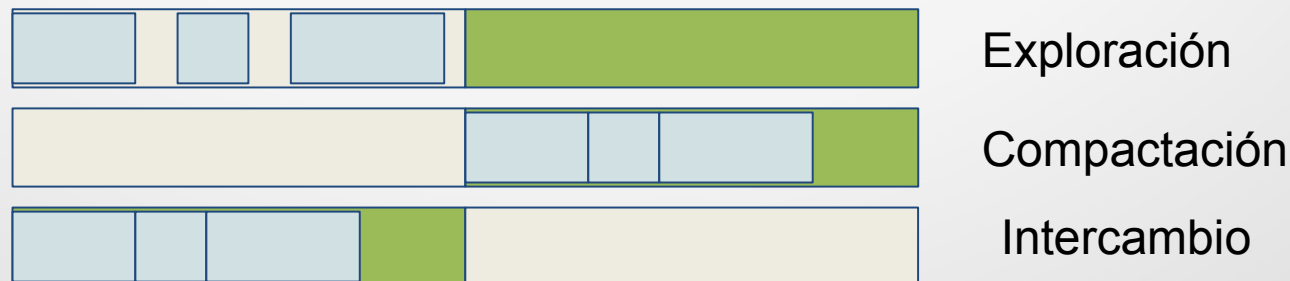
TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

Estrategias y Versiones.

Stop and Copy

Al trabajar con bloques de tamaño variable en la heap, en el momento de liberar la memoria esta podría quedar fragmentada, por lo que su reutilización se puede ver comprometida. Podríamos reducir esta fragmentación realizando una compactación de almacenamiento. El método stop-and-copy divide la heap en dos regiones de igual tamaño. Toda la asignación ocurre en la primera mitad. Cuando esta mitad está (casi) llena, el recolector comienza su exploración de estructuras de datos alcanzables. Cada bloque alcanzable se copia en la segunda mitad, sin fragmentación externa. Cuando el recolector termina su exploración, todos los objetos útiles han sido movidos (y compactados) a la segunda mitad de la heap, y nada en la primera mitad se necesita más. El recolector puede, por lo tanto, intercambiar su noción de primera y segunda mitad, y el programa puede continuar. Obviamente, este algoritmo sufre del hecho de que solo la mitad del montón puede ser utilizada en cualquier momento, sin embargo puede usarse un mecanismo de memoria virtual para mitigar este defecto.



TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

Estrategias y Versiones.

Recolección Generacional

Para reducir aún más el costo de la recolección, algunos recolectores de basura emplean una técnica "generacional", aprovechando la observación de que la mayoría de los objetos asignados dinámicamente tienen vida corta. La Heap se divide en múltiples regiones (a menudo dos). Cuando el espacio escasea, el recolector primero examina la región más joven, que supone tiene la mayor proporción de basura. Sólo si no puede reclamar suficiente espacio en esta región, el recolector examina la siguiente región más antigua. En la mayoría de los casos, la sobrecarga de la recolección será proporcional sólo al tamaño de la región más joven.

Cualquier objeto que sobreviva a un pequeño número de recolecciones (a menudo una) en su región actual es promovido (movido) a la siguiente región más antigua, de una manera similar a stop-and-copy. La promoción requiere, por supuesto, que los punteros de objetos antiguos a objetos nuevos se actualicen para reflejar las nuevas ubicaciones.

TIPOS DE DATOS – MANEJO DE MEMORIA

● Liberación de Memoria - Garbage Collector – Comparación de Implementaciones

	Java	PHP	Python	Javascript
Mecanismo	Generacional	Cuenta de Referencias	Cuenta de Referencias (opcional: recolector de basura cíclico generacional).	En versiones viejas utilizaba conteo por referencias.
Reconocimiento	Divide la Heap en 3 generaciones (Joven / Vieja y Permanente)	Contenedor z-val con 2 campos: is_ref y refcount		
Algoritmo	Se van envejeciendo y compactando los objetos referenciados. Se eliminan objetos sin referencia y se organizan los bloques de generaciones.	Se cuenta la cantidad de referencias de cada objeto y al llegar a 0, si no es referenciado se elimina	Proporciona una interfaz para el recolector de basura opcional con la capacidad de deshabilitarlo, ajustar la frecuencia de recolección y establecer opciones de depuración.	En la actualidad marca y barrido pero lo define su implementación (web-browser, motor de ejecución, etc)
Configuraciones	Serial GC – Parallel GC - CMS GC – Garbage First	Fallo con fuga de memoria solucionado a partir de 5.3.0		

TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

El caso Rust

Al asignar un objeto de memoria en la heap se considera lo siguiente.

- La memoria debe solicitarse al administrador de memoria en tiempo de ejecución.
- Necesitamos una forma de devolver esta memoria al administrador cuando terminemos de trabajar con el objeto.

En los lenguajes con un recolector de basura (Garbage Collector), el recolector de basura rastrea y limpia la memoria que ya no se está usando y no necesitamos pensar en ello ya que generalmente se ejecuta automáticamente. En la mayoría de los lenguajes sin un recolector de basura, es nuestra responsabilidad identificar cuándo la memoria ya no se está usando y llamar al código para liberarla explícitamente.

Rust toma un camino diferente: la memoria se devuelve automáticamente una vez que la variable que la posee sale del contexto de ejecución.

TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

El caso Rust

Mantener un registro de qué partes del código están utilizando qué datos en el heap, minimizar la cantidad de datos duplicados en el heap y limpiar los datos no utilizados en el heap para que no se quede sin espacio son todos problemas que el concepto de ownership (en rust) aborda.

TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

El caso Rust

La memoria se administra a través del concepto de **propiedad (ownership)**. El ownership refiere a un conjunto de reglas a través de las cuales se administra la memoria en Rust.

Reglas de Ownership

- Cada valor en Rust tiene un propietario (una variable).
- Solo puede haber un propietario a la vez.
- El propietario de un dato puede prestarlo o entregar la propiedad del mismo.
- Cuando el propietario sale del alcance, el valor se descartará.

Un contexto de ejecución es el rango o espacio dentro de un programa para el que un elemento es válido (ámbito).

Para los tipos de datos de tamaño conocido en compilación utilizará el stack (o memoria de la pila) sin embargo para aquellos objetos que no conoce su tamaño y que puede ser dinámico utilizará la HEAP.

TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

El caso Rust

```
{ let s1= String::from("hola");
```

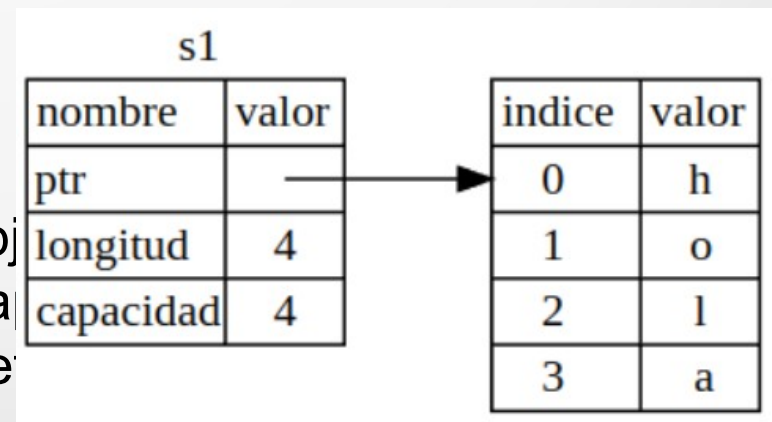
```
let s2= s1;
```

```
println!("{}", s1);} //DARÁ UN ERROR al intentar imprimir s1 ya que movió su  
representación a s2.
```

Un Objeto de dato está compuesto por tres partes:

- Un puntero a la memoria que contiene el contenido de la cadena (se almacena en Heap),
- Longitud,
- Capacidad (se almacena en Stack).

Cuando asignamos s1 a s2, los datos de Obj que copiamos el puntero, la longitud y la capacidad, pero no copiamos los datos en el heap al que hace referencia.

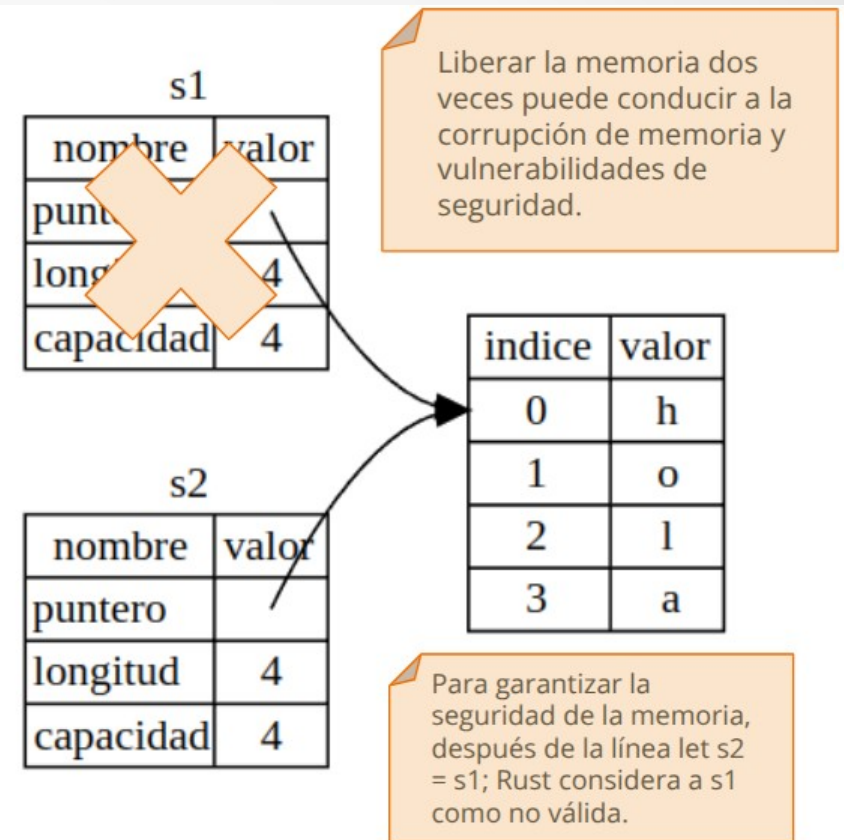


TIPOS DE DATOS – MANEJO DE MEMORIA

Garbage Collector

El caso Rust

Rust no permite que dos punteros apunten al mismo lugar para evitar errores de doble liberación. Se llama a la función `drop` automáticamente para `s1`. Si el valor es de longitud fija e inmutable, puede ser copiado automáticamente y no movido. Si se trata de usar la variable “movida”, ocurrirá un error en compilación.



TIPOS DE DATOS

TIPO DE DATO ABSTRACTO

TIPOS DE DATOS - TADs

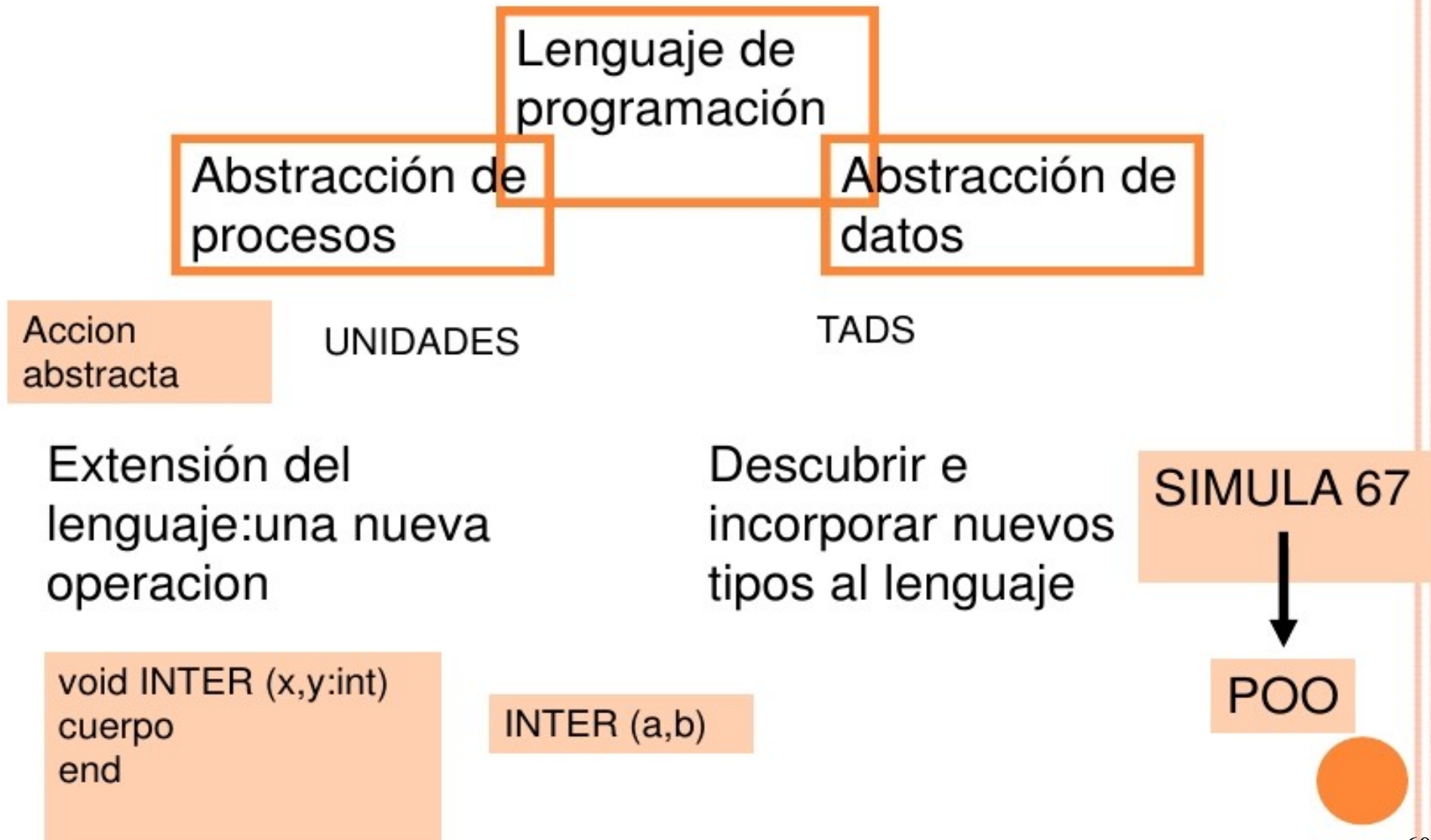
ABSTRACCIÓN

La abstracción es el mecanismo que tenemos las personas para manejar la complejidad

Abstraer es representar algo descubriendo sus características esenciales y suprimiendo las que no lo son.

El principio básico de la abstracción es la **información oculta**.

TIPOS DE DATOS - TADs



TIPOS DE DATOS - TADs

TAD

**TAD = Representación (datos) + Operaciones
(funciones y procedimientos)**

Los tipos de datos son abstracciones y el proceso de construir nuevos tipos se llama **abstracción de datos**.

Los nuevos tipos de datos definidos por el usuario se llaman **tipos abstractos de datos**.

TIPOS DE DATOS - TADs

TAD

Tipo abstracto de dato (TAD) es el que satisface:

- Encapsulamiento: la representación del tipo y las operaciones permitidas para los objetos del tipo se describen en una única unidad sintáctica.

Refleja las abstracciones descubiertas en el diseño

- Ocultamiento de la información: la representación de los objetos y la implementación del tipo permanecen ocultos.

Refleja los niveles de abstracción. Modificabilidad

TIPOS DE DATOS - TADs

TAD

Las unidades de programación de lenguajes que pueden implementar un TAD reciben distintos nombres:

- Simula-67 proporciona una estructura sintáctica que permite que las operaciones y la representación puedan especificarse en una única unidad sintáctica (class). Pero no satisface el ocultamiento de la información.
- Modula-2 módulo
- Ada paquete
- C++ clase
- Java clase

TIPOS DE DATOS - TADs

ESPECIFICACIÓN DE UN TAD

La especificación formal proporciona un conjunto de axiomas que describen el comportamiento de todas las operaciones. Ha de incluir una parte de sintaxis y una parte de semántica Por ejemplo:

TAD nombre del tipo (valores que toma los datos del tipo)

Sintaxis

Operación(Tipo argumento, ...) -> Tipo resultado

....

Semántica

Operación(valores particulares argumentos) $\Rightarrow$ expresión
resultado

Hay operaciones definidas por sí mismas que se consideran constructores del TAD. Normalmente, se elige como constructor la operación que inicializa.

TIPOS DE DATOS - TADs

EJEMPLO TAD: CONJUNTO

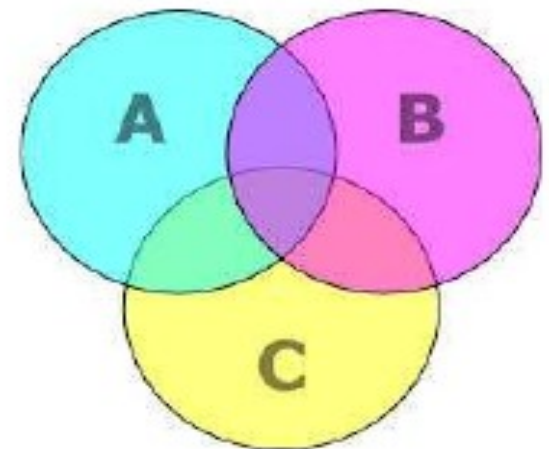
- *TAD Conjunto (colección de elementos sin duplicidades, pueden estar en cualquier orden, se usa para representar los conjuntos matemáticos con sus operaciones).*

Sintaxis

- $*\text{Conjuntovacio} \rightarrow \text{Conjunto}$
- $*\text{Añadir}(\text{Conjunto}, \text{Elemento}) \rightarrow \text{Conjunto}$
- $\text{Retirar}(\text{Conjunto}, \text{Elemento}) \rightarrow \text{Conjunto}$
- $\text{Pertenece}(\text{Conjunto}, \text{Elemento}) \rightarrow \text{boolean}$
- $\text{Esvacio}(\text{Conjunto}) \rightarrow \text{boolean}$
- $\text{Cardinal}(\text{Conjunto}) \rightarrow \text{entero}$
- $\text{Union}(\text{Conjunto}, \text{Conjunto}) \rightarrow \text{Conjunto}$

Semántica $\forall e1, e2 \in \text{Elemento}$ y $\forall C, D \in \text{Conjunto}$

- $\text{Añadir}(\text{Añadir}(C, e1), e1) \Rightarrow \text{Añadir}(C, e1)$
- $\text{Añadir}(\text{Añadir}(C, e1), e2) \Rightarrow \text{Añadir}(\text{Añadir}(C, e2), e1)$
- $\text{Retirar}(\text{Conjuntovacio}, e1) \Rightarrow \text{Conjuntovacio}$
- $\text{Retirar}(\text{Añadir}(C, e1), e2) \Rightarrow \text{si } e1 = e2 \text{ entonces } \text{Retirar}(C, e2)$
sino $\text{Añadir}(\text{Retirar}(C, e2), e1)$



TIPOS DE DATOS - TADs

EJEMPLO TAD: PILA EN ADA

- PILA de enteros (100)

ESPECIFICACION

- package PILA IS
- type PILA limited private
- MAX: constant := 100
- function EMPTY (P:in PILA) return boolean
- procedure PUSH (P:inout PILA,ELE:in INTEGER)
- procedure POP (P: inout PILA)
- procedure TOP (P:inPILA) return INTEGER

ENCAPSULA

- private
- type PILA is
- vecpila : array (1..MAX) of INTEGER
- tope: INTEGER range 0..MAX:=0
- end PILA

OCULTA

TIPOS DE DATOS - TADs

IMPLEMENTACION

```
○ package body PILA is
○   function EMPTY (P:in PILA) return boolean
○       .....
○   end
○   prodedure PUSH (P:inout PILA,ELE:in INTEGER)
○       .....
○   end
○   procedure POP (P: inout PILA)
○       .....
○   end
○   procedure TOP (P:inPILA) return INTEGER
○       .....
○   end
○ end PILA
```

OCULTA



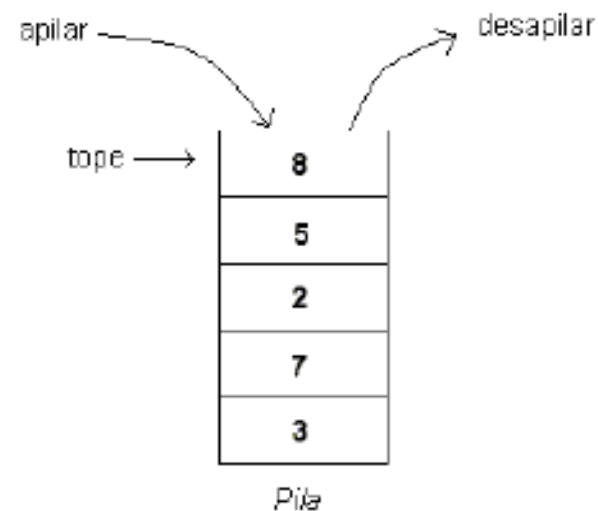
TIPOS DE DATOS - TADs

INSTANCIACION DE UNA PILA

```
with PILA
procedure USAR
  pil:PILA
  y: INTEGER
  .....
  pil.PUSH (pil,y)
End USAR
```

INSTANCIA
ALOCA Y EJECUTA
EL CODIGO DE
INICIALIZACION

APILA



TIPOS DE DATOS - TADs

TAD: CLASES

En términos prácticos, una clase es un tipo definido por el usuario

Una clase contiene la especificación de los datos que describen un objeto junto con la descripción de las acciones que un objeto conoce.

Atributos + Métodos

Agrega un segundo nivel de abstracción que consiste en agrupar las clases en jerarquías de clases. De forma que la clase hereda todas las propiedades de la superclase.

TIPOS DE DATOS - TADs

EJEMPLO: CLASE EN JAVA

```
class Punto
{ private int x; // coordenada x
  private int y; // coordenada y
  public Punto(int x_, int y_) // constructor
  { x = x_; y = y_; }
  public Punto() // constructor sin argumentos
  { x = y = 0; }
  public int leerX() // devuelve el valor de x
  { return x; }
  public int leerY() // devuelve el valor de y
  { return y; }
  void fijarX(int valorX) // establece el valor de x
  { x = valorX; }
  void fijarY(int valorY) // establece el valor de y
  { y = valorY; }
}
```

Visibilidad de los miembros de la clase

```
Punto p;
p = new Punto();
p.fijarX (100);
System.out.println(" Coordenada x es " + p.leerX());
```

TIPOS DE DATOS

SISTEMA DE TIPOS

TIPOS DE DATOS – SISTEMA DE TIPOS

SISTEMA DE TIPOS

Conjunto de reglas usadas por un lenguaje para estructurar y organizar sus tipos.

El objetivo de un sistema de tipos es escribir programas seguros.

Conocer el sistema de tipos de un lenguaje nos permite conocer de una mejor forma los aspectos semánticos del lenguaje.

TIPOS DE DATOS – SISTEMA DE TIPOS

SISTEMA DE TIPOS

- Provee mecanismos de expresión:
 - Expresar tipos intrínsecos o definir tipos nuevos.
 - Asociar los tipos definidos con construcciones del lenguaje.
- Define reglas de resolución:
 - Equivalencia de tipos – ¿dos valores tienen el mismo tipo?.
 - Compatibilidad de tipos – ¿puedo usar el tipo en este contexto?
 - Inferencia de tipos – ¿cuál tipo se deduce del contexto?
- Mientras más flexible el lenguaje, más complejo el sistema

SEGURIDAD VS. FLEXIBILIDAD

TIPOS DE DATOS – SISTEMA DE TIPOS

Esto es una definición genérica de fuertemente tipado.

BIL

Se dice que el sistema de tipos es fuerte cuando especifica restricciones sobre como las operaciones que involucren valores de diferentes tipos pueden operarse. Lo contrario establece un sistema débil de tipos.

```
a = 2
b = "2"
Concatenar (a,b) //retorna "22"
Sumar (a,b) //retorna 4
```

```
a = 2
b = "2"
Concatenar (a,b) //error de tipos
Sumar (a,b) //error de tipos
Concatenar (str(a),b) //retorna "22"
Sumar (a,int(b)) //retorna 4
```


TIPOS DE DATOS – SISTEMA DE TIPOS

SISTEMA DE TIPOS - ESPECIFICACIÓN

- Tipo y tiempo de chequeo
- Reglas de equivalencia y conversión
- Reglas de inferencia de tipo
- Nivel de polimorfismo del lenguaje

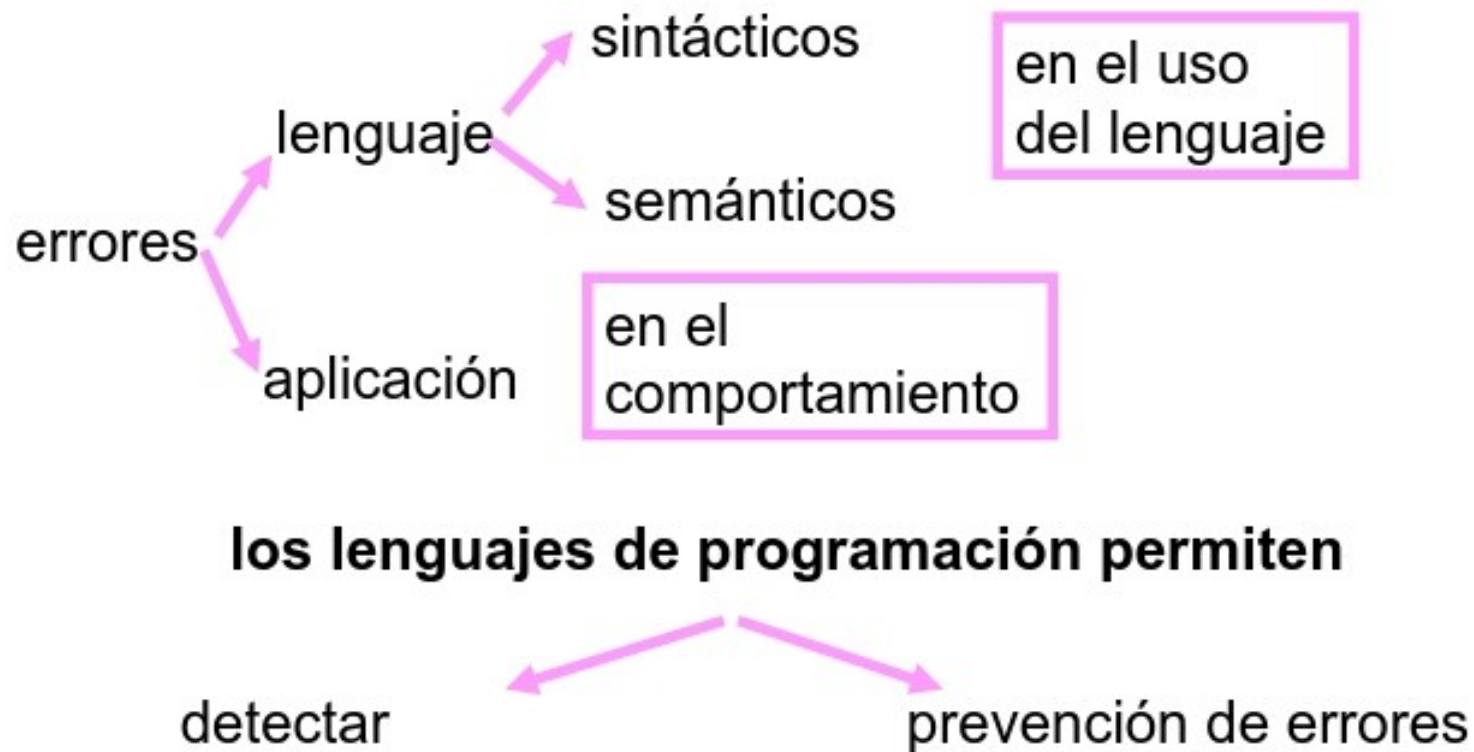
TIPOS DE DATOS – SISTEMA DE TIPOS

Tipo y Tiempo de Chequeo

Tipos de ligadura

- Tipado estático: ligaduras en compilación. Para esto puede exigir:
 - Se puedan utilizar tipos de datos predefinidos
 - Todas las variables se declaran con un tipo asociado
 - Todas las operaciones se especifican indicando los tipos de los operandos requeridos y el tipo del resultado.
- Tipado dinámico: ligaduras en ejecución, provoca mas comprobaciones en tiempo de ejecución (no es seguro???)

TIPOS DE DATOS – SISTEMA DE TIPOS



TIPOS DE DATOS – SISTEMA DE TIPOS

Lenguaje Fuertemente Tipado

- Si el lenguaje es fuertemente tipado el compilador puede garantizar la ausencia de errores de tipo en los programas (Ghezzi).
- Un lenguaje se dice fuertemente tipado (type safety) si el sistema de tipos impone restricciones que aseguran que no se producirán errores de tipo en ejecución.
- Un lenguaje se dice fuertemente tipado (type safety) si todos los errores de tipo se detectan.

TIPOS DE DATOS – SISTEMA DE TIPOS

Lenguaje Fuertemente Tipado

Definición:

Esta definición está básicamente orientada a lenguajes compilados

Un lenguaje se dice fuertemente tipado (type safety) si el sistema de tipos impone restricciones que aseguran que no se producirán errores de tipo en ejecución.

En esta concepción, la intención es evitar los errores de aplicación y son tolerados los errores del lenguaje, como los errores de compilación.

Python es Fuertemente Tipado y tiene tipado dinámico.
C es débilmente tipado y tiene tipado estático.
GOBSTONE Fuertemente Tipado y tipado estático.

TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Equivalencia y Conversión

- Tipo compatible: reglas semánticas que determinan si el tipo de un objeto es válido en un contexto particular
- Un lenguaje debe definir en qué contexto el tipo Q es compatible con el tipo T.
- Si el sistema de tipos define la compatibilidad.

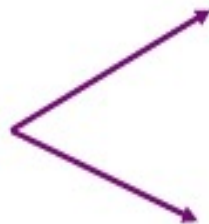
TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Equivalencia y Conversión

```
type  t = array [1..100] of integer  
var   x,y : array [1..100] of integer  
z: array [1..100] of integer  
w:t  
v:t
```

w, x, y, z, v son del mismo tipo??

Compatibilidad



Nombre

Estructura

TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Equivalencia y Conversión

- Equivalencia por nombre: dos variables son del mismo tipo si y solo si están declaradas juntas o si están declaradas con el mismo nombre de tipo.
- Equivalencia por estructura: dos variables son del mismo tipo si los componentes de su tipo son iguales.

TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Equivalencia y Conversión

- Un tipo es compatible con otro si
 - es equivalente
 - se puede convertir
- Coerción: significa convertir un valor de un tipo a otro. Reglas del lenguaje de acuerdo al tipo de los operandos y a la jerarquía

TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Equivalencia y Conversión

- Widening (ensanchar): cada valor del dominio tiene su correspondiente valor en el rango. (Entero a Real). Pascal solo widening de entero a real.
- Narrowing (estrechar): cada valor del dominio puede no tener su correspondiente valor en el rango. En este caso algunos lenguajes producen un mensaje avisando la pérdida de información. (Real a Entero). En C Depende del contexto y utiliza un sistema de coersión simple.
- Cláusula de casting : conversiones explícitas, se fuerza a que se convierta.

TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Inferencia y nivel de polimorfismo

La inferencia de tipos permite que el tipo de una entidad declarada se “infiera” en lugar de ser declarado. La inferencia puede realizarse de acuerdo al tipo de:

- Un operador predefinido

fun f1(n,m)=(n mod m=0)

- Un operando

fun f2(n) = (n*2)

- Un argumento

fun f3(n) = n*n

- El tipo del resultado

fun f4(n) = (n*n)

TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Inferencia y nivel de polimorfismo

Veamos los siguientes ejemplos:

```
write(e,f(x)+1)
```

```
fun disjuntos(s1,s2:Conjunto):boolean;
```

- La función disjuntos deberá implementarse para cada tipo particular de conjunto?
- Las funciones read y write son “polimórficas”, pero no de forma pura (ya que el compilador infiere el tipo)

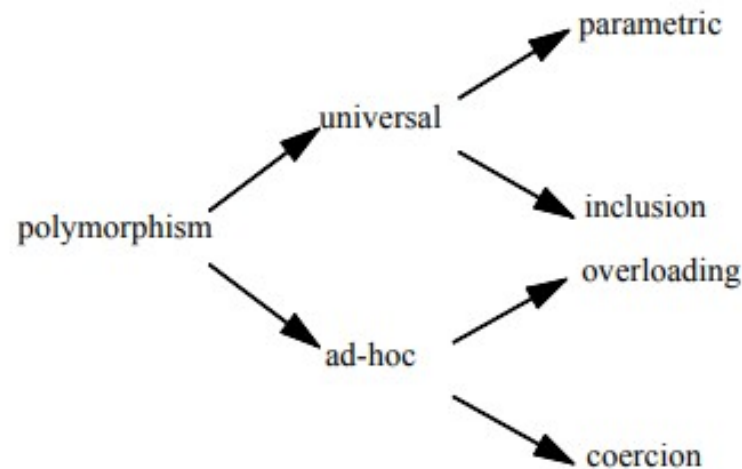
TIPOS DE DATOS – SISTEMA DE TIPOS

- Un lenguaje se dice mono-mórfico si cada entidad se liga a un único tipo (estáticos). En un lenguaje de programación mono-mórfico la función disjuntos deberá implementarse para cada tipo particular de conjunto.
- Un lenguaje se dice **polimórfico** si las entidades pueden estar ligadas a más de un tipo
- Las variables polimórficas pueden tomar valores de diferentes tipos
- Las operaciones polimórficas son funciones que aceptan operandos de varios tipos
- Los tipos polimórficos tienen operaciones polimórficas

TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Inferencia y nivel de polimorfismo

Todos los lenguajes prácticos tienen cierto grado de polimorfismo. En consecuencia, las preguntas importantes a responder son: ¿Qué diferentes tipos (o grados) de polimorfismo se identifican? ¿Hasta dónde podemos llegar?



TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Inferencia y nivel de polimorfismo

El **polimorfismo ad-hoc** permite que una función se aplique a distintos tipos con un comportamiento sustancialmente diferente en cada caso

El término **sobrecarga** se utiliza para referirse a conjuntos de abstracciones diferentes que están ligadas al mismo símbolo o identificador.

La **coerción** permite que un operador que espera un operando de un determinado tipo T puede aplicarse de manera segura sobre un operando de un tipo diferente al esperado

TIPOS DE DATOS – SISTEMA DE TIPOS

Reglas de Inferencia y nivel de polimorfismo

El **polimorfismo universal** permite que una única operación se aplique uniformemente sobre un conjunto de tipos relacionados

Si la uniformidad de la estructura de tipos está dada a través de parámetros, hablamos de **polimorfismo paramétrico**

Un tipo parametrizado es un tipo que tiene otros tipos como parámetros. Ejemplo Tipo Lista con Tipo T como parámetro.

$\text{lista}(T) = T^*$

El **polimorfismo por inclusión** es otra forma de polimorfismo universal que permite modelar subtipos y herencia.

- Si un tipo se define como un conjunto de valores y un conjunto de operaciones. Un subtipo T' de un tipo T puede definirse como un subconjunto de los valores de T y el mismo conjunto de operaciones.

- El mecanismo de subtipos puede ser definido a partir de una clase base ya existente. Podría a

```
subtype TDíaDelMes is Integer range 1..31;  
subtype TDíaFebrero is TDíaDelMes range 1..29;  
subtype TLaborable is TDíaDeSemana range Lunes..Viernes;
```

partir de una clase base ya